



Ravi Sharma <ravikr845430@gmail.com>

Fwd: Interview Prep-Google

1 message

Richa Verma <vricha211697@gmail.com>
To: Ravi Sharma <ravikr845430@gmail.com>

Sat, Aug 3, 2024 at 2:11 PM

----- Forwarded message -----

From: **Shivraj Pavaskar (xWF)** <shivrajp@xwf.google.com>
Date: Mon, Jul 29, 2024 at 3:01 PM
Subject: Interview Prep-Google
To: <vricha211697@gmail.com>

Hi Richa ,

Warm Greetings from Google India !

It was a pleasure speaking with you! :)

To get up to speed with respect to how and what to prepare, please find below certain materials/links which will be extremely useful to you. Do have a thorough look at them.

Overall process:**Recruiter Prescreen → Onsite Interview (5 sessions) → Hiring Committee Review → Offer Review → Offer Delivery (Woohoo!)****Step 1. Refresh yourself with CS fundamentals**

- We do expect you to know a lot about algorithms and data structures and especially be able to implement them into your solutions - there is a great [biggocheatsheet](#) that may also help you!
- [Steve Yegge's Blog](#)
- Google Style Guides ([C++](#), [Python](#), [Java](#), [Android](#), [Javascript](#))
- [Geek for Geeks](#) - Study algorithms and data structure
- System Design resources:
 - [System design interview tip!](#)
 - [Hired in Tech](#)

Step 2. Coding Practice + CS review

When you practice, do not use an IDE. You need to be able to write legible, compilable code without help with regards to layout, or spelling of standard library class/method names. I suggest solving similar style algorithmic/ DS problems on a google document or on paper to simulate a real interview. Several sites that provide similar problems to those typically asked in the interview are:

- [Leetcode](#)
- [Code Jam Kickstart](#) - great practice to get a glimpse of [Google coding and algorithm questions](#). Code Jam hosts online Kickstart rounds that give participants the opportunity to test and grow their coding abilities! Participate in [one](#)—or join them all! You can also learn how to solve those problems by reading our [analysis](#)
- [Codeforces](#)
- [HackerRank](#)

- [Topcoder](#)
- [ACM-ICPC](#)
- [Code Jam](#)

Additional Notes/ Optional Studies:

- **Study Materials:**
 - **Courses: can choose one of below**
 - [Coursera - Algorithms, Part 1](#)
 - [Coursera - Algorithms, Part 2](#)
 - [Udacity - Intro to Algorithms](#)
 - [MIT Open courseware - Introduction to Algorithms](#) (not efficient)
 - **Youtube Channels:**
 - [Tushar Roy - Coding made Simple](#)
 - [Interview Prep](#)
 - [Google Recruiters Share Technical Interview Tips](#) (30 min)
 - [Google Technical Interview Workshop - Singapore](#) (2 hours):
 - **Books:**
 - [Beautiful code](#)
 - [Elements of Programing Interview](#)
 - [Coding Interview University](#)
 - [Cracking the Coding Interview](#)

Understand Google Interview Process

- [Ask a Google Engineer — What is the Interview Process at Google?](#) (2 min)
- [How to prepare for a Google Engineering Interview](#) (7 min)
- [Candidate Coaching Session for Technical Interviewing](#) (45 min)
- [Example of a Coding Interview](#) (24 min)

Initial Interview (DS/Algo):

- Duration: 45mins
- Google hangouts call (video is optional)
- Coding is done on a Virtual Interviewing Platform, similar to Google Docs.

You will be expected to come up with optimized and production-ready code, free of bugs. You should not write in pseudo-code. I would **highly recommend** watching this [video](#) first to get a good grasp of the tips/ideas/areas of focus for the interviews here at Google.

Important areas of assessment

- **Coding** - Good code quality (No IDE. Fully functional code. Take care of syntax, good language constructs, clean and concise code) & coding fast (you can code in Codechef timed challenges to practise coding under time pressure)
- **DS & Algo** - Refer to below topics on what we are assessing for
- **Attitude** - Being independent, lead/drive the discussion forward while being receptive to hints from interviewers and integrating it in your solution
- **Speed of solving the question** (Ideally 20mins per question - refer to below notes on time allocation)

Strategy for Success

- Step 1: Clarify the problem
- Step 2: Define your approach and propose a solution
- Step 3: Code and implement a fully working solution. Cover for edge/boundary cases

- Step 4: Dry run (sample Test Cases) along with discussion with the interviewer, fixing bugs

This is simply a recommendation but does not need to be followed for every question.

Breakdown of Framework

- **Tip 1: Think out Loud & Communicate your Thought Process**
 - Google interviewers are assessing you based on your problem solving skills, and want to know your full thought process behind why and how you derived the final result
 - **Talk through your entire thought process & explain about your approach**
 - Most important would be to pick the right data structure and algorithm for a specific problem! I
 - **Do not** mumble to yourself or keep silent when thinking.
- **Tip 2: Ask Clarifying Questions (Very Important!)**
 - Never jump straight and code up the solution. Always take a pause, look at the interview question and ask some clarifying questions
- **Tip 3: Discuss algorithmic complexities & Identify all Edge Cases Independently**
 - For Algorithms, you will need to know big-o notation very well.
 - **Always state time and space complexities upfront. Think of how you can reduce the complexity further to reach an optimised solution!**
 - Distinguish between average case/worst case runtime
 - Consider amortized time complexities!
 - **The goal is to reach the most optimised solution** at the end of the interview, and to have a complete working solution.
- **Tip 4: Test your Code**
 - **Check for boundary conditions!**
 - Stress tests to ensure that the code runs within time constraint
 - Create tests that have 100% code coverage
 - Rectify any bugs in your code before the interviewer points it out
- **Tip 5: Ensure Good Code Quality on Google Docs**
 - You are expected to code in Google Docs. Since it does not have any IDE, you are expected to type out your code from scratch
 - Type as close to fully functional code as possible. This code should be **maintainable and readable by a large database of engineers**
 - **Do not** use Pseudo code or shortcuts, it is not good enough.

Interview Preparation Plan

- **Revise all concepts on data structures & algorithms- you can also use [this gitHub link](#) on CS fundamentals that can serve as a checklist while preparing. This BigO cheat sheet <<http://bigocheatsheet.com/>> could help you as well!**
- **Structured Revision plan** on the topics that to cover (Eg. hashtable, hashmaps, trees, arrays, strings, graphs, dynamic programming and more)
 - **Practise per category**
 - **Practice up to a level that you reach competency**
- **Practice Problem Identification** by picking random problems and practice identifying "Which category does this problem belong to? Backtracking/Dynamic programming? Solution/Algorithmic design?"
- **Practice coding without an IDE**, be familiar with the differences in how you should write code in Google Docs. Do practise coding in Google Docs within a set time frame and getting comfortable talking while coding.

In summary: Practise a wide variety of questions, and simulate actual interview conditions! You can also run mock interviews here at pramp.com

Frequently asked topics (in no particular order)

- Binary search
- BFS/DFS/Flood fill

- Tree traversals
- Hash tables
- Linked list, stacks, queues, two pointers/sliding window
- Binary heaps
- Dynamic programming
- Union find
- Ad hoc/string manipulations
- Other good to know topics: Trie, segment trees/fenwick trees, bitmask
- Google Interview Style Guides ([C++](#), [Python](#), [Java](#), [Javascript](#))
- You'll be expected to know and apply: **lists, maps, stacks, priority queues, binary trees, graphs, bags, and sets.**
- For algorithms you'll want to know **greedy algorithms, divide and conquer, dynamic programming, recursion, and brute force search.**
- You'll definitely want to be conversant with bigO notation, time-space complexity, and real world performance of all of this.

What we are assessing for Data Structures & Algorithms

- Can you implement the most optimized data structure and algorithm for the question?
- Can you explain the tradeoffs between the data structure/solution?
- Can you explain why you choose a data structure for implementation
- Can you explain and analyze the time and space complexity correctly
- Can you translate the algorithm to code well?

**Build
for
everyone**

Shivraj Pavaskar
Staffing Specialist- Randstad Sourceright supporting Google
M: +914067412849



[|Learn more about our candidate privacy policy.|](#)

"Google is a disability and neurodiversity inclusive employer. If you require any accommodations throughout the recruitment process, please reach out to your global disability partner, [EmployAbility](#), for a confidential discussion. [Schedule a call](#) with our accommodations expert or email employability.org@gmail.com for additional support."

If you received this communication by mistake, please don't forward it to anyone else (it may contain confidential or privileged information), please erase all copies of it, including all attachments, and please let the sender know it went to the wrong person. Thanks..